

Homework 3

Natasha Atkins

Exploring Looping with Air Quality Data

Part 1: Build the dataset

```
#load libraries  
library(tidyverse)
```

Warning: package 'ggplot2' was built under R version 4.4.3

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.1.4      v readr      2.1.5  
v forcats    1.0.0      v stringr    1.5.1  
v ggplot2    4.0.2      v tibble     3.2.1  
v lubridate  1.9.3      v tidyr      1.3.1  
v purrr      1.0.2  
-- Conflicts ----- tidyverse_conflicts() --  
x dplyr::filter() masks stats::filter()  
x dplyr::lag()     masks stats::lag()  
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(janitor)
```

Attaching package: 'janitor'

The following objects are masked from 'package:stats':

chisq.test, fisher.test

```
library(ggplot2)

#create pm2.5 data
set.seed(1)
pm25 = runif(min=0, max=50, n=365)
total = 0
nvalues = 365
pm25
```

```
[1] 13.2754332 18.6061950 28.6426682 45.4103895 10.0840966 44.9194842
[7] 47.2337634 33.0398896 31.4557022 3.0893135 10.2987287 8.8278376
[13] 34.3511423 19.2051859 38.4920710 24.8849621 35.8809254 49.5953047
[19] 19.0017590 38.8722611 46.7352616 10.6071261 32.5836883 6.2777548
[25] 13.3610334 19.3057046 0.6695167 19.1193979 43.4845423 17.0174498
[31] 24.1040058 29.9782913 24.6770654 9.3108801 41.3686659 33.4233369
[37] 39.7119930 5.3971813 36.1855473 20.5637215 41.0473147 32.3530097
[43] 39.1466381 27.6518156 26.4859790 39.4678116 1.1665601 23.8615033
[49] 36.6156869 34.6365778 23.8809811 43.0604738 21.9048554 12.2398639
[55] 3.5339524 4.9733080 15.8135854 25.9317132 33.1002538 20.3415094
[61] 45.6437962 14.6801686 22.9532863 16.6197337 32.5435234 12.9008390
[67] 23.9272624 38.3155335 4.2123457 43.7660665 16.9536469 41.9720175
[73] 17.3341745 16.6887465 23.8175623 44.6099168 43.2169735 19.4994772
[79] 38.8660349 48.0308999 21.7329742 35.6257339 19.9997184 16.2676076
[85] 37.8543574 10.1346128 35.5560611 6.0845961 12.2744257 7.1652190
[91] 11.9814708 2.9467189 32.1144129 43.8134606 38.9457339 39.8654413
[97] 22.7637227 20.5042041 40.5435121 30.2466645 32.7361964 17.6598636
[103] 13.5130073 49.6342031 31.6746632 10.6604068 6.4686174 23.9059017
[109] 46.2037235 29.9380484 48.8085347 36.5896256 17.8363456 21.5736845
[115] 7.4105780 0.6538788 35.7783033 5.1592118 22.3142174 32.0050523
[121] 49.5919310 24.7796789 24.2174762 8.6721167 37.7410472 22.6947745
[127] 25.5584892 10.3772557 11.4329071 29.7855998 28.7436099 3.8532190
[133] 1.7770290 32.1397746 46.4307600 29.9046211 28.0450374 26.3013862
[139] 49.2547612 25.3820911 34.1394039 30.0770609 11.9434339 12.9082963
[145] 36.4654812 22.6285416 8.7563384 37.3349135 5.2493820 43.2272475
[151] 30.7322486 27.8579769 16.4388660 22.6565723 25.0220486 9.0433181
[157] 26.4815301 3.7637873 13.8877966 10.6349760 14.2395240 44.7547051
[163] 22.3117662 38.9992445 44.0309517 20.6562105 3.1904240 16.7743746
[169] 36.1862973 16.8807667 31.5207061 42.0307277 42.8065832 19.5679641
[175] 19.0246943 44.7722713 32.2157881 37.0539324 30.2651723 45.1540806
[181] 14.6865078 9.5630055 44.3225472 25.1669743 43.8528772 9.4596811
[187] 37.9051526 36.2249446 47.1862409 27.3823294 35.5871934 19.4452550
[193] 5.0436563 46.3651044 14.1616250 29.5286579 5.5180302 42.0253516
```

```

[199] 15.8981842 39.1425668 13.3754104 10.9322642 25.8398418 13.4475296
[205] 9.0584164 25.9288069 28.1391468 6.4578427 12.8183802 35.8967638
[211] 48.0704968 5.0070423 38.1611345 47.3983177 40.9317344 15.4146165
[217] 32.4789730 47.6677726 47.6866325 16.9989602 13.1237055 8.2726967
[223] 16.1084028 25.5062603 46.1984236 25.5479849 12.8810630 2.3230443
[229] 20.8928129 42.7000751 17.3615339 6.5721160 18.7243432 31.5710114
[235] 19.5039467 34.4813924 34.4706706 27.7450312 21.4812204 22.6360031
[241] 15.3221629 28.9176972 45.5185152 7.1302041 20.7523813 10.5462875
[247] 21.4375185 6.6344988 23.0048223 47.1478530 38.0986931 46.6454914
[253] 23.5339249 30.1794034 24.2494840 5.4403158 12.3863416 24.9257265
[259] 18.6433354 46.7345685 26.1993039 15.8572336 13.8983015 39.3770254
[265] 35.1231256 8.2513819 3.2228769 37.7352811 31.0205017 8.4788383
[271] 3.1107026 5.4514634 19.0858176 8.4655457 14.9326271 9.6104768
[277] 12.8585011 9.0615911 23.8656855 38.5368521 1.3893561 26.3655388
[283] 44.0159534 18.6531686 2.3979566 6.9314123 16.0746060 7.7415806
[289] 6.6114086 11.0652964 11.3190398 6.5708267 49.0781730 16.3506863
[295] 25.3469749 34.0721257 4.9584552 5.9451279 2.5219830 46.4626960
[301] 33.6856116 4.7428928 24.6298060 23.0775920 18.7608265 49.5549610
[307] 8.8175357 40.6717604 3.4223319 20.0224873 7.0572163 9.6654931
[313] 42.0675858 35.9956994 13.3606042 24.7500822 4.1556949 17.6942120
[319] 48.4604403 31.2357095 33.2309125 15.6244828 20.2844806 49.8038686
[325] 42.7541178 47.6774198 40.6152546 39.1091058 13.3939064 38.1075765
[331] 49.3155795 14.6802775 19.9675553 40.6065762 3.8575835 18.1848405
[337] 22.1296234 7.8357066 29.1102635 48.5081089 49.4749917 8.8226018
[343] 27.1065212 19.2151946 33.8082025 13.4646890 23.4625471 8.5900041
[349] 18.4594731 36.2702636 24.3074552 3.1901233 39.2273115 20.9160818
[355] 49.0509042 14.1441978 42.3941075 4.1119615 44.3229375 23.5965365
[361] 5.4550482 16.6638992 41.8708285 13.8424921 29.3517571

```

```

#days
day = (1:365)

#seasons
season = ifelse(day <= 90, "winter",
               ifelse(day <= 180, "spring",
               ifelse(day <= 270, "summer", "fall")))
season = as.factor(season)

#risk
risk = rep(NA, 365)

#build data frame

```

```
aq_dataframe = data.frame(day=day, pm25=pm25, season=season, risk=risk )
head(aq_dataframe)
```

```
  day    pm25 season risk
1   1 13.27543 winter  NA
2   2 18.60619 winter  NA
3   3 28.64267 winter  NA
4   4 45.41039 winter  NA
5   5 10.08410 winter  NA
6   6 44.91948 winter  NA
```

```
summary(aq_dataframe)
```

```
      day          pm25          season      risk
Min.   : 1   Min.   : 0.6539   fall   :95   Mode:logical
1st Qu.: 92   1st Qu.:13.2754   spring:90   NA's:365
Median :183   Median :23.8615   summer:90
Mean   :183   Mean   :24.7107   winter:90
3rd Qu.:274   3rd Qu.:36.4655
Max.    :365   Max.    :49.8039
```

Part 2: Classify risk with a for loop

```
#classify the thresholds
```

```
#good_aq <= 12
```

```
#moderate_aq > 12 & <= 35.4
```

```
#unhealthy_aq > 35.4
```

```
ndays = 365
```

```
#initializing the variables
```

```
max_unhealthy_streak = 0
```

```
current_streak = 0
```

```
#the beginning of the loop says to look through the rows (days) and for every pm value in the
```

```
#if the values are classified as good or moderate then they don't contribute to the unhealthy
```

```
for (i in 1:ndays){
  if(aq_dataframe[i,"pm25"] <= 12) {
```

```

    aq_dataframe[i, "risk"] = "good"
    current_streak = 0
  } else {
    if (aq_dataframe[i, "pm25"] <= 35.4){
      aq_dataframe[i, "risk"] = "moderate"
      current_streak = 0
    } else {
      aq_dataframe[i, "risk"] = "unhealthy"
      current_streak = current_streak + 1
    }
  }
  if (current_streak > max_unhealthy_streak){
    max_unhealthy_streak = current_streak
  }
}
print(max_unhealthy_streak)

```

[1] 5

Part 3: Seasonal summary with a for loop

```

# initialize vector
seasonal_unhealthy = c(winter = 0, spring = 0, summer = 0, fall = 0)

# iterate by looking at each row for each day
for (i in 1:ndays){
  # check for unhealthy day --> and if the day is unhealthy check the season
  if (aq_dataframe[i, "risk"] == "unhealthy"){
    if (aq_dataframe[i, "season"] == "winter"){
      # count the day toward the correct season
      seasonal_unhealthy[1] = seasonal_unhealthy[1] + 1
    } else if (aq_dataframe[i, "season"] == "spring"){
      seasonal_unhealthy[2] = seasonal_unhealthy[2] + 1
    } else if (aq_dataframe[i, "season"] == "summer"){
      seasonal_unhealthy[3] = seasonal_unhealthy[3] + 1
    } else {
      seasonal_unhealthy[4] = seasonal_unhealthy[4] + 1
    }
  }
}
}

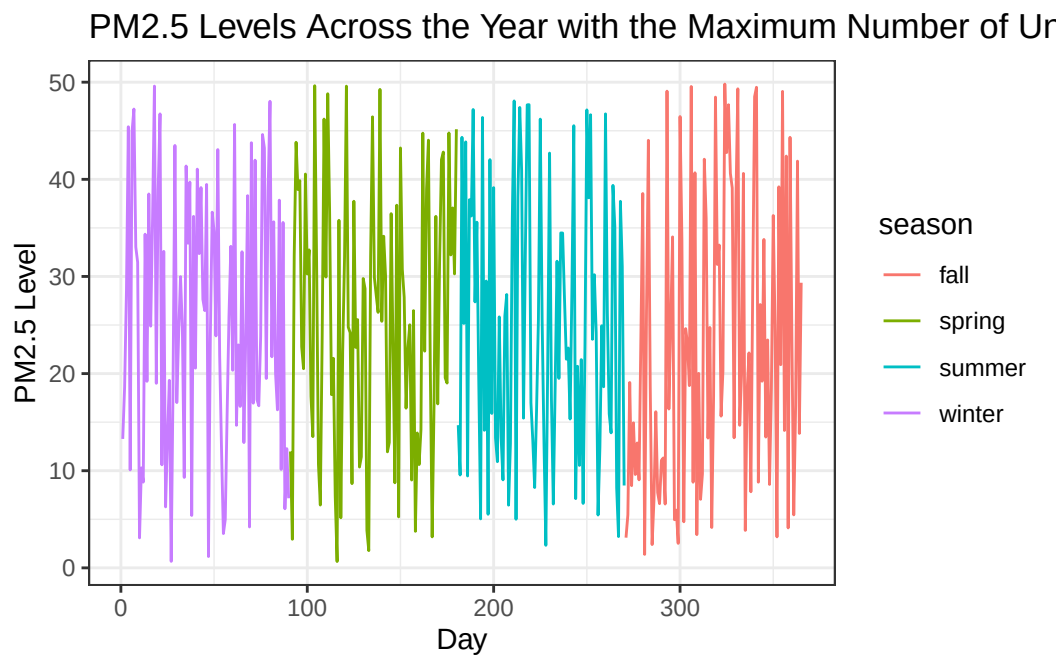
```

```
print(seasonal_unhealthy)
```

```
winter spring summer fall
      28      25      25      25
```

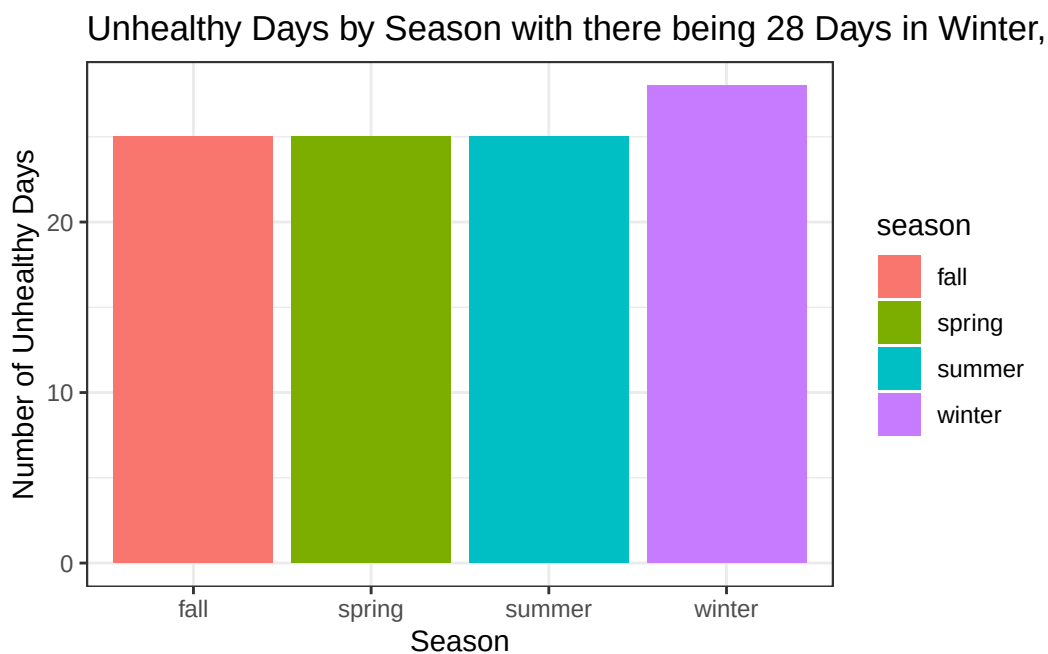
Part 4: Visualize the data with plots

```
#time series of pm2.5 values
aq_plot <- ggplot(aq_dataframe, aes(x = day, y = pm25, color = season)) +
  geom_line() +
  theme_bw() +
  labs(
    title = sprintf("PM2.5 Levels Across the Year with the Maximum Number of Unhealthy Days I"),
    x = "Day",
    y = "PM2.5 Level"
  )
aq_plot
```



```
# bar chart of unhealthy days per season
# build a data frame to access the number of unhealthy days for the plot
seasonal_unhealthy_df <- data.frame(
  season = names(seasonal_unhealthy),
  count = as.numeric(seasonal_unhealthy)
)

ggplot(seasonal_unhealthy_df, aes(x = season, y = count, fill = season)) +
  geom_col() +
  theme_bw() +
  labs(
    title = sprintf("Unhealthy Days by Season with there being %d Days in Winter, %d Days in
                     seasonal_unhealthy[\"winter\"], seasonal_unhealthy[\"spring\"],
                     seasonal_unhealthy[\"summer\"], seasonal_unhealthy[\"fall\"]",
    x = "Season",
    y = "Number of Unhealthy Days"
  )
)
```



Summary – Interpretation for Managers

The major patterns revealed from this analysis are that the longest streak of unhealthy days was 5 days in a row with unhealthy air quality (PM2.5 levels above 35.4 $\mu\text{g}/\text{m}^3$). This

unhealthy streak occurred during the summer on days 210-215 of the year. The season with the highest number of unhealthy days total was the winter with 28 unhealthy days. All 3 other seasons had 25 days of unhealthy air quality. While all seasons had relatively similar amount of unhealthy days (as is expected with the uniform distribution we set to generate the data), summer having 3 more days than other seasons may indicate that scheduling outdoor activities, such as construction, land management, recreational events, might be safer to do not in the summer.